

Full Platform

10-Day Delivery Workbook

Developer 03 — Platform, Security & Release

Owns infrastructure, security, Authentik, Docker, CI/CD, staging/production releases, backups, and observability.

Delivery control	Agreed working rule
Programme target	Complete the full CERA first-release platform in ten intense working days. Work is sliced vertically and integrated daily.
Branch base	All feature branches start from develop. main is production-only. No direct pushes to protected branches.
Required approvals	One PR approval minimum; two for authentication, database migrations, secrets, deployment, or security changes.
Definition of launch	Public services/blog, CMS, secure enquiry, customer status, staff workflow, Zoho, Resend, Authentik, R2, monitoring, backup and documented deployment all work in staging and production.

1. Recommended project decisions

Decision	Recommended direction
Customer journey	Browse public services and blog posts; submit a secure service enquiry; optionally create an account to view enquiry status and updates. Online payment is not in v1 until CERA selects a payment provider in writing.
Staff roles	Customer; Content Editor; Operations/Support; Administrator. Authentik enforces role-based access.
Data boundary	Collect only name, email, phone, selected service, message, consent, and enquiry status. Do not collect diagnosis, medical history, prescriptions, or health documents in v1.
Zoho records	Create/update Lead or Contact with full name, email, phone, selected service, source, consent timestamp, enquiry ID, owner, and status.
Content approval	CERA nominates one Content & Clinical Approver. No health claim, service description, blog, or public pricing is published without that person's written approval.
Release approval	Dev 3 approves technical readiness. CERA's nominated Product Owner approves staging/UAT. Production needs both approvals in GitHub.
Account ownership	CERA owns and pays all vendor accounts: Hetzner, Cloudflare, R2, GitHub, Resend, Zoho, domain, and backups. Developers receive only necessary role-based access.
Jurisdiction	Swabi, Khyber Pakhtunkhwa, Pakistan.

2. Your ownership and boundaries

Area	Your responsibility
Primary modules	Docker Compose; Hetzner CX43/CX33; Caddy; Cloudflare; GitHub Actions; Authentik; R2; GlitchTip; backups; release controls.
Key integrations	Provides secure deployment/runtime for Dev 1 and Dev 2 applications.
Review focus	Secrets, access rights, container hardening, health checks, data backups, migrations, release rollback, and incident response.

3. Daily collaboration routine

- 09:30 — 15-minute stand-up: yesterday, today, blockers, files likely to conflict.
- Before coding — read the assigned issue, acceptance criteria, and shared contract/type.
- During coding — work only on the named branch; do not edit another developer's active module without agreement.
- Before PR — run lint, type-check, relevant tests, and manual local verification.
- PR review — one approval minimum; two approvals for auth, database migrations, secrets, deployment, or security changes.
- End of day — update the GitHub Issue, post staging link/test evidence where available, and name any hand-off needed tomorrow.

4. Ten-day build schedule

Each day ends with a pushed branch, updated issue, test evidence, and a clear hand-off. If a task cannot be completed, cut non-critical polish—not security, data ownership, test coverage, or release controls.

Day	Branch	Build work	Done	Hand-off
1	chore/infra-bootstrap	Create Client-owned accounts/projects; configure CX43 production and temporary CX33 staging; private networks, firewall baseline, SSH keys, Docker, and Caddy.	Both environments reachable through approved access only.	Never put credentials in GitHub issues, code, or chat.
2	ci/github-actions-pipeline	Create CI: install, lint, type-check, tests, build, Docker image, dependency checks. Configure protected main/develop and GitHub Environments.	PRs cannot merge without green checks.	Dev 1/2 add their test commands to CI.
3	feat/authentik-roles	Configure Authentik tenants/apps/clients, MFA for admins, roles for customer/editor/operations/admin, and login callback routes.	Role-based staging login works with test accounts.	Dev 1 integrates login; Dev 2 checks API claim validation.
4	feat/cloudflare-r2-caddy	Configure Cloudflare DNS/TLS, Caddy reverse proxy/security headers, R2 buckets/credentials, upload permissions, and no public DB/Valkey ports.	Staging uses HTTPS and protected service boundaries.	Confirm media flow with Dev 1.
5	feat/staging-deployment	Automate develop → staging deployment, image tags, health checks, migration sequence, and rollback to previous image.	Merged staging PR deploys predictably and smoke tests pass.	Dev 2 supplies migration command; Dev 1 supplies smoke tests.
6	feat/backup-restore	Enable Hetzner backups; add encrypted PostgreSQL backup to R2; document retention; perform a restore into staging.	Restore evidence and recovery steps documented.	No production health/patient data is used for development testing.
7	feat/glitchtip-observability	Configure GlitchTip projects, alert routing, structured logs, uptime/health endpoint checks, and release/version reporting.	A deliberate test error reaches GlitchTip with release version.	Each developer verifies errors are meaningful, not secret-bearing.
8	test/security-release-	Run access review,	Technical release gate is	Log all release blockers in

Day	Branch	Build work	Done	Hand-off
	checks	secret scan, dependency audit, firewall review, rate-limit/security-header checks, and disaster-recovery rehearsal.	either approved or blocked with clear issues.	GitHub.
9	fix/uat-release	Support UAT fixes, verify image reproducibility, finalise production environment, and prepare rollback command/owner.	Production release checklist ready for CERA approval.	Only release-critical changes are accepted.
10	release/production-v1.0.0	Approve GitHub production deployment, create tag, deploy, run smoke tests, monitor 24 hours, and hand over access/runbook.	Release record, rollback reference, and support handover completed.	Merge any hotfixes back to develop.

5. Integration contracts and hand-offs

Direction	Working agreement
Read from Dev 1	Required routes, public/protected pages, image upload behaviour, and browser smoke tests.
Read from Dev 2	Service health endpoints, migration commands, background jobs, database and Valkey requirements, webhook paths.
Provide to team	Environment contract, staging URL, GitHub Environment rules, deployment logs, backup runbook, and release checklist.

6. Git and pull-request rules

- Use conventional commits: feat(scope): description; fix(scope): description; test(scope): description; ci: description.
- Keep PRs single-purpose. Include issue link, screenshots for UI work, tests run, migration impact, and rollback note where relevant.
- Before PR: git fetch origin; git rebase origin/develop; resolve conflicts; run the relevant tests; push only your branch.
- Never commit .env files, passwords, API keys, backups, real customer data, or production database dumps.
- A database migration must be reversible or backward-compatible and reviewed by Dev 2 and Dev 3.

7. Day-10 release and post-launch responsibility

- Freeze new feature work after Day 8. Only UAT defects, security fixes, and release documentation are allowed.
- Create release/v1.0.0 from develop; deploy it to staging; run the complete smoke-test checklist.
- CERA Product Owner records staging approval in the release issue; Dev 3 records technical approval.
- Merge release/v1.0.0 to main, create annotated tag v1.0.0, and deploy the immutable image.
- Watch GlitchTip, health checks, logs, Resend delivery, R2 access, and Zoho sync for the first 24 hours.
- If a critical issue appears, create hotfix/<name> from main, test, release, and merge the fix back to develop.

Named developer: _____ Date assigned: _____